



Systemes Numériques / FPGA

SÉBASTIEN VAN
CAUWENBERGHE

(S3V@ECAM.BE)

Sommaire



Lesson 1. Introduction aux FPGA et logique combo



Lesson 2. Logique sequentielle



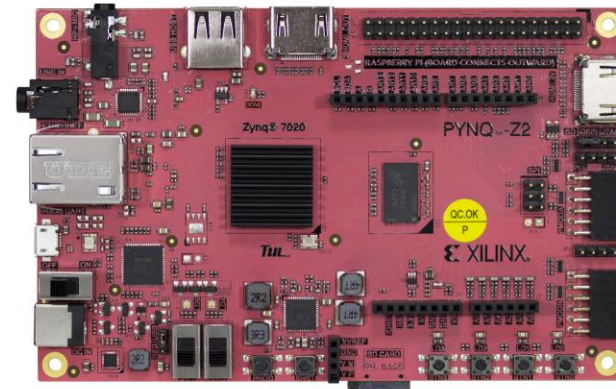
Lesson 3. FSM



Lesson 4. Interfaces



Lesson 5. Pipelines



Introduction

FPGA et Zynq:

- Process en VHDL
- Logique Séquentielle / Synchrone
- FSM
- Demo



Cours 2 : Récap

- Logique séquentielle
- FSM design



How to get started to design

Input:

- Spécification système

Output :

- FPGA specification
- FPGA architecture description
- RTL design
- Pin allocation
- Timing constraints
- Bitstream
- Synthesis log files

Specification

Input:

- Spécification système
- Lister les interfaces
- Lister chaque fonctionnalité
- Lister les performances attendues pour chaque fonctionnalité

Exercice : Ecrire la spec FPGA pour un convertisseur de puissance à controle numérique

Specification - Exercice

Exercice : Ecrire la spec FPGA pour un convertisseur de puissance à controle numérique

- Le système a 1 ADC SPI à 1 MHz
- Le convertisseur est constitué d'un half bridge
- La tension de sortie doit être régulée

Interface ?

Fonctionnalités ?

Performance ?

Architecture

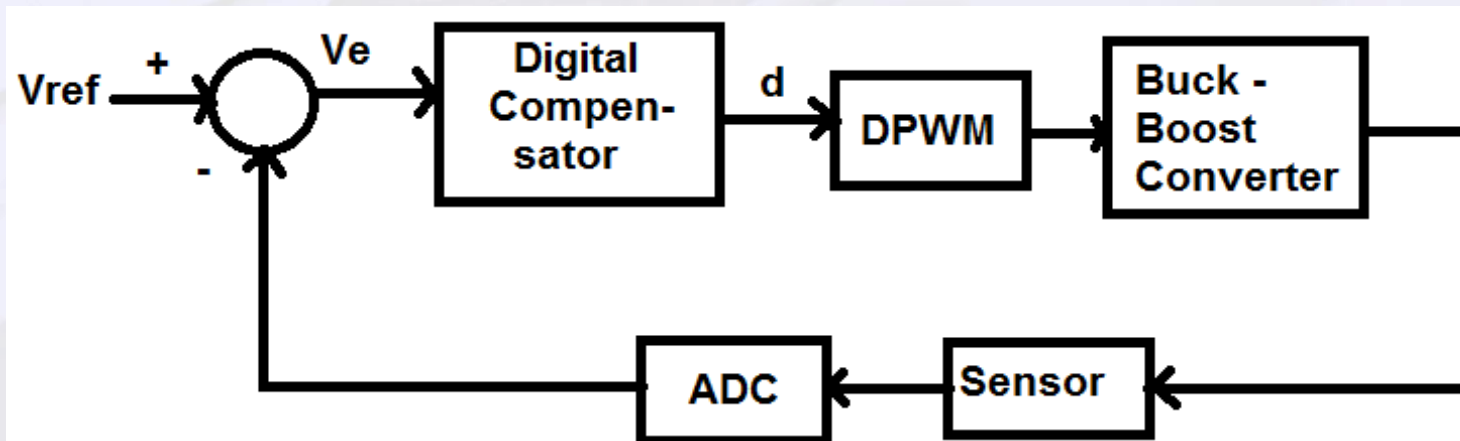
Input:

- Spécification FPGA
- Découper en blocs fonctionnels
- Verifier la dépendance entre les blocs
- Connecter les blocs (comment ?)
- Configuration ?

Architecture - Exercice

Exercice : Déterminer l'architecture

- 1 Contrôleur ADC SPI
- 1 PI
- 1 PWM complémentaire



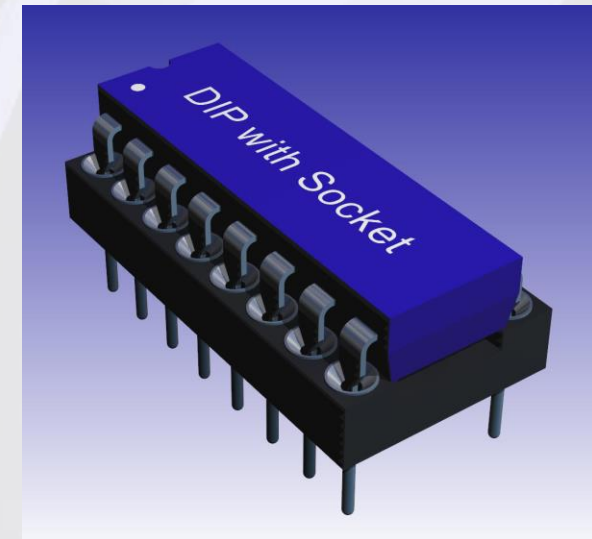
VHDL

- Plusieurs révisions : 1993, 2002, 2008, 2017 (presque pas utilisable)
- Typage fort
- Si ça compile ça peut marcher, si ça compile pas, ça marche pas :)

VHDL - Intro

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
  
entity first_design is  
  port(  
    a : in  std_logic;  
    b : in  std_logic;  
    z : out std_logic  
  );  
end entity first_design;  
  
architecture RTL of first_design is  
  
begin  
  z <= a and b;  
end architecture RTL;
```

- Component : Prototype
- Entity : Ports d'entrée/sortie du module
- Architecture : Description du module



VHDL - Adder

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity full_adder is
  port(
    a : in  std_logic;
    b : in  std_logic;
    cin : in std_logic;
    y : out std_logic;
    cout : out std_logic
  );
end entity full_adder;

architecture RTL of full_adder is
  signal half_sum : std_logic;
  signal half_cy : std_logic;

begin
  half_sum <= a xor b;
  y <= half_sum xor cin;

  half_cy <= a and b;
  cout <= half_cy or (half_sum and cin);
end architecture RTL;
```

Exercice :

- Ecrire la table de vérité
- Dessiner le schéma en portes de ce circuit

VHDL – Assignment

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_1 is
  port(
    in1      : in  signed(7 downto 0);
    in2      : in  signed(7 downto 0);
    max      : out signed(7 downto 0);
    biggest  : out std_logic;
    const    : out std_logic_vector(3 downto 0)
  );
end entity mux_1;

architecture RTL of mux_1 is

begin
  max      <= in1 when in1 >= in2 else in2;
  biggest  <= '0' when in1 >= in2 else '1';
  const    <= X"5" when in1 = in2
            else X"A" when not biggest
            else 4X"F";
end architecture RTL;
```

Assignation conditionnelle

<signal> <= <valeur vraie> [when <condition> else
<valeur fausse>]

VHDL – With/Select

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_2 is
  port(
    in1      : in std_logic_vector(1 downto 0);
    gray_out : out std_logic_vector(1 downto 0)
  );
end entity mux_2;

architecture RTL of mux_2 is
begin
  with in1 select gray_out <=
    "00" when "00",
    "01" when "01",
    "11" when "10",
    "10" when others;

end architecture RTL;
```

Construction

With <signal> Select <sortie> <= xxx when <condition> ,

....

Yyyy when others;

VHDL - Instantiation

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity max_4 is
    port(
        in1      : in  signed(7 downto 0);
        in2      : in  signed(7 downto 0);
        in3      : in  signed(7 downto 0);
        in4      : in  signed(7 downto 0);
        max       : out signed(7 downto 0)
    );
end entity max_4;
```

```
architecture RTL of max_4 is
    signal max12 : signed(7 downto 0);
    signal max34 : signed(7 downto 0);
begin
    max_12 : entity work.mux_1
        port map(
            in1    => in1,
            in2    => in2,
            max     => max12,
            biggest => open,
            const  => open
        );
```

```
    max_34 : entity work.mux_1
        port map(
            in1    => in3,
            in2    => in4,
            max     => max34
        );

    max_end : entity work.mux_1
        port map(
            in1    => max12,
            in2    => max34,
            max     => max
        );
end architecture RTL;
```

<label> : entity work.<nom de l'entité>
Port (
 <port a l'intérieur> => <port a l'exterieur>
);

VHDL - types

Types en VHDL :

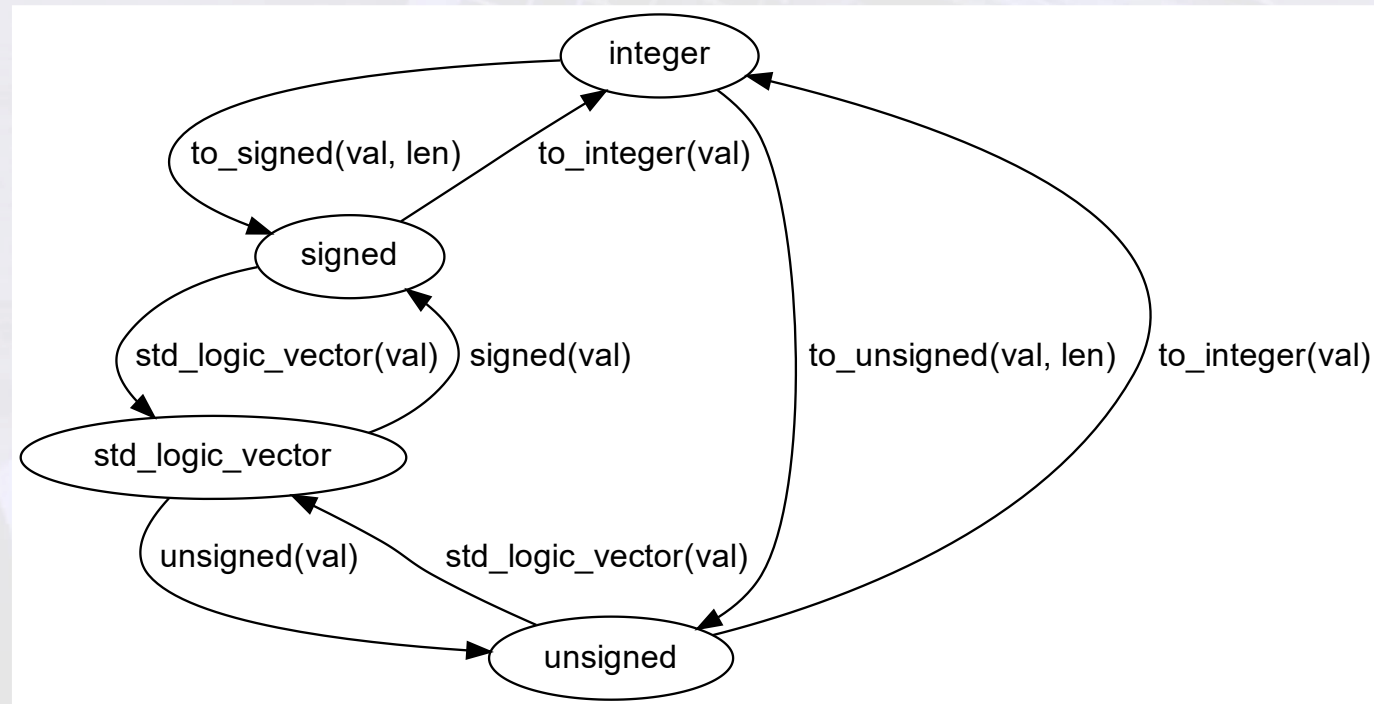
- Boolean
- Std_logic: 1 bit (0, 1, Z, X, U, H, L, -)
- Std_logic_vector, signed, unsigned : ensemble de n éléments de type std_logic.
- La notation (m downto n), indique un vecteur de m-n+1 bits. Souvent n=0
- Integer : nombre signé de 32 bits (voire 64 actuellement)

Exemple :

Signal av : unsigned (31 downto 0);

VHDL – Type conversion

Types en VHDL : conversions



VHDL - Operators

Concaténation : `out <= a & b;`

Selection d'un vecteur : `out <= in(3 downto 0);`

Selection d'un bit : `out <= in(2);`

Opérateurs arithmétiques :

			Synth
+ (a,b)	Addition	Longueur (a)	V
- (a,b)	Soustraction	Longueur (a)	V
* (a,b)	Multiplication	Longueur (a) + Longueur (b)	V
/ (a,b)	Division	Longueur (a)	X
Mod (a, b)	Modulo	Longueur (a)	X
Rem (a, b)	Remainder	Longueur (a)	X

VHDL – Operators 2

Opérateurs arithmétiques :

Opération	Description	Valeur de retour	Synth
- a	Opposé de a	Longueur (a)	V
Abs(a)	Valeur absolue	Longueur (a)	V

Opérateurs de comparaison :

Opérateur	Description	Valeur de retour	Synth
>	Plus grand que	boolean	V
<	Plus petit que	boolean	V
>=	Plus grand ou egal	boolean	V
<=	Plus petit ou egal	boolean	V
=	Egal	boolean	V
/=	Différent	boolean	V

VHDL – Operators 3

Operateurs de logiques :

Opérateur	Description	Valeur de retour	Synth
and/nand	ET, NON ET	vecteur	V
or/nor	OU, NON OU	vecteur	V
not	NOT	vecteur	V
xor/xnor	OU EXCLUSIF, NON OU EXCLUSIF	vecteur	V

Opérateurs de shift

Opérateur	Description	Valeur de retour	Synth
sla/sra	Shift Arithmetique	vecteur	V
sll/srl	Shift Logique	vecteur	V
rol	Rotation Gauche	vecteur	V
ror	Rotation Droite	vecteur	V

VHDL – Signal example

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity operator_demo is
  port(
    a : in  unsigned(7 downto 0);
    b : in  unsigned(7 downto 0);
    z : out unsigned(7 downto 0);
    m_low : out unsigned(7 downto 0);
    m_high : out unsigned(7 downto 0)
  );
end entity operator_demo;

architecture RTL of operator_demo is
  signal multiply : unsigned(15 downto 0);
begin
  z <= a + b;
  multiply <= a * b;
  m_low <= multiply(7 downto 0);
  m_high <= multiply(15 downto 8);
end architecture RTL;
```

Signal : fil entre 2 points (peut être de n'importe quel type)

On doit calculer manuellement la taille du résultat de la multiplication

La multiplication a une taille de la somme des opérandes

VHDL – Constants

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_1 is
  port(
    in1      : in  signed(7 downto 0);
    in2      : in  signed(7 downto 0);
    max      : out signed(7 downto 0);
    biggest  : out std_logic;
    const    : out std_logic_vector(3 downto 0)
  );
end entity mux_1;

architecture RTL of mux_1 is
begin
  max      <= in1 when in1 >= in2 else in2;
  biggest  <= '0' when in1 >= in2 else '1';
  const    <= X"5" when in1 = in2
            else X"A" when not biggest
            else 4X"F";
end architecture RTL;
```

1 bit

'0', '1', 'Z', 'X', 'U', ...

multi bits :

Binaire : "0011010" = b"0011010" = 7b"0011010"

Octal : o"755" = 3o"755" <=> b"111101101".

Longueur de 3 bits par digit.

Hexa : x"5A" = 7X"5A" <=> "01011010".

Longueur de 4 bits par digit

Process en VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity demo_sensitivity is
  port(
    input_1 : in std_logic;
    input_2 : in std_logic;
    output : out std_logic
  );
end entity demo_sensitivity;

architecture RTL of demo_sensitivity is

begin
  sel_p : process(all) is
  begin
    output <= input_1 when input_2 else '0';
  end process sel_p;
end architecture RTL;
```

- Liste de sensibilité, n'active le process lors du changement de certains signaux

Process en VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity demo_case is
  port(
    opc : in std_logic_vector(1 downto 0);
    a : in std_logic;
    b : in std_logic;
    output : out std_logic
  );
end entity demo_case;

architecture RTL of demo_case is
begin
  sel_p : process(opc, a, b) is
  begin
    case opc is
      when "00" =>
        output <= a and b;

      when "01" =>
        output <= a or b;

      when "10" =>
        output <= a xor b;

      when others =>
        output <= not a;
    end case;
  end process sel_p;
end architecture RTL;
```

- Case <var> is
 - When
<value> =>
<statements>
 -
 - When others
=> <statements>
- End case;

Process en VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity demo_if is
    port(
        a      : in  std_logic;
        b      : in  std_logic;
        output : out std_logic
    );
end entity demo_if;

architecture RTL of demo_if is

begin
    if_p : process(a, b) is
    begin
        output <= '1';
        if a then
            output <= '0';
        elsif b then
            output <= '0';
        end if;

    end process if_p;

end architecture RTL;
```

- If <condition>
then
- <statements>
- End if;

Process en VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity demo_loop is
  port(
    a      : in  std_logic_vector(15 downto 0);
    output : out std_logic_vector(15 downto 0)
  );
end entity demo_loop;

architecture RTL of demo_loop is
begin
  loop_p : process(all) is
  begin
    for i in 0 to 15 loop
      output(i) <= a(15-i);
    end loop;

  end process loop_p;
end architecture RTL;
```

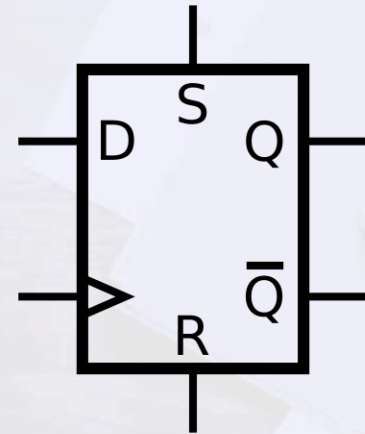
- For <var> in <L>
 <to|downto>
 <R> loop
 - <statements>
 - End loop;
-
- <statements>
 - End loop;
-
- While
 <condition>
 loop (not
 synthesizable
)

D Flip-flop

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dff is
  port(
    clk : in  std_logic;
    rst : in  std_logic;
    d   : in  std_logic;
    q   : out std_logic
  );
end entity dff;

architecture RTL of dff is
begin
  dff_p : process(clk) is
  begin
    if rising_edge(clk) then
      if rst then
        q <= '0';
      else
        q <= d;
      end if;
    end if;
  end process dff_p;
end architecture RTL;
```

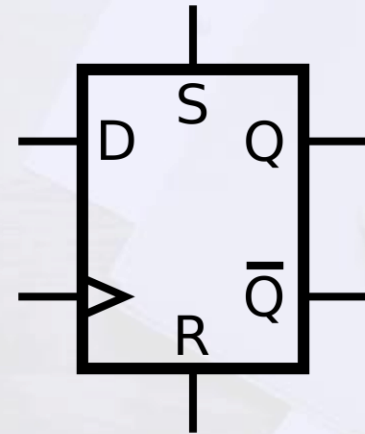


D Flip-flop (async reset)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dff_async is
  port(
    clk : in  std_logic;
    rst : in  std_logic;
    d   : in  std_logic;
    q   : out std_logic
  );
end entity dff_async;

architecture RTL of dff_async is
begin
  dff_p : process(clk, rst) is
  begin
    if rst then
      q <= '1';
    elsif rising_edge(clk) then
      q <= '0';
    end if;
  end process dff_p;
end architecture RTL;
```



Logic design example

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter is
  port(
    clk      : in  std_logic;
    Counter_o : out std_logic_vector(7 downto 0)
  );
end entity counter;

architecture RTL of counter is
  signal counter_i : unsigned(7 downto 0);
begin
  cntr_p : process(clk) is
  begin
    if rising_edge(clk) then
      if counter_i < 220 then
        counter_i <= counter_i + 1;
      else
        counter_i <= (others => '0');
      end if;
    end if;
  end process;

  Counter_o <= std_logic_vector(counter_i);

end architecture RTL;
```

- If : Fonctionne comme un multiplexeur

Logic design example

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dff_sync is
  port(
    clk : in  std_logic;
    i   : in  std_logic;
    o   : out std_logic
  );
end entity dff_sync;

architecture RTL of dff_sync is
  signal sync : std_logic_vector(1 downto 0);

begin
  dff_p : process(clk) is
  begin
    if rising_edge(clk) then
      sync(0) <= i;
      sync(1) <= sync(0);
    end if;

    o <= sync(1);
  end process dff_p;

end architecture RTL;
```

Logic design example

Rising Edge detection

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity edge_detect is
  port(
    clk : in  std_logic;
    i   : in  std_logic;
    o   : out std_logic
  );
end entity edge_detect;

architecture RTL of edge_detect is
  signal i_d : std_logic;

begin
  dff_p : process(clk) is
  begin
    if rising_edge(clk) then
      i_d <= i;
    end if;
  end process dff_p;
  o <= i and not i_d;

end architecture RTL;
```


Logic design example

Shift register, Parallel/Serial converter

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity shift_reg is
  port(
    clk      : in  std_logic;
    s_i      : in  std_logic;
    s_o      : out std_logic;
    p_i      : in  std_logic_vector(7 downto 0);
    p_i_write : in  std_logic;
    p_o      : out std_logic_vector(7 downto 0)
  );
end entity shift_reg;

architecture RTL of shift_reg is
  signal sreg : std_logic_vector(7 downto 0) := (others => '0');

begin
  dff_p : process(clk) is
  begin
    if rising_edge(clk) then
      sreg <= sreg(6 downto 0) & s_i;
      if p_i_write then
        sreg <= p_i;
      end if;
    end if;
  end process dff_p;
  s_o <= sreg(7);
  p_o <= sreg;

end architecture RTL;
```